



Unsupervised Steering Vectors from Learned Classifiers: A Summary

October 8, 2026

WHAT THE METHOD DOES

The method, which we call an *Ontologizer*, rewrites a neural network's internal representation at one point as a short list of discrete choices, and rebuilds the representation from them. It is made of small classifiers. Each classifier sorts every input into one of k categories, and each category stores one vector; the input is rebuilt as the sum of the vectors its categories select. In the models described here there are 5 rounds of 32 classifiers with 32 categories each, so an input is described by 160 choices — 800 bits — plus one number per round giving its overall size. Nothing is labeled: classifiers and vectors are learned only by asking the rebuilt representation to match the original.

Because a classifier's categories are mutually exclusive, changing one classifier's choice for an input — from category j to category j' — swaps one stored vector for another. That swap is the steering move the method is built around: it moves the representation from j to j' along one axis the method found, and the result is always another valid description, so the edited representation stays within the range the network normally sees.

We study two representations. The main one is GPT-2 small's internal state after its first eight blocks [1], on web text: there the rebuilt state can be put back into the running model, so we can measure how much of the model's behavior survives. The second is SONAR sentence embeddings [2], a fixed-length vector per sentence that can be decoded back into text, which makes a steered embedding readable.

WHAT IT IS FOR

To steer a model is to change its internal state so that it behaves differently in a chosen way. The most reliable current method is a *steering vector*: average the

representations of examples with a property, subtract the average of examples without it, and add the difference to new inputs [3, 4]. This works well, but each concept needs a labeled contrastive dataset, and someone must already suspect the concept exists.

Sparse autoencoders (SAEs) find features without labels: each input is described by a few active features out of thousands [5, 6]. But each feature is an isolated direction with no stated alternatives, features split as the dictionary grows, and adding a feature direction can push the representation anywhere; on standard steering benchmarks, simple supervised baselines beat them [7].

The goal is steering vectors that need no labels but keep the useful shape of the supervised kind: each one a move between named alternatives, found together with the contrast it moves along.

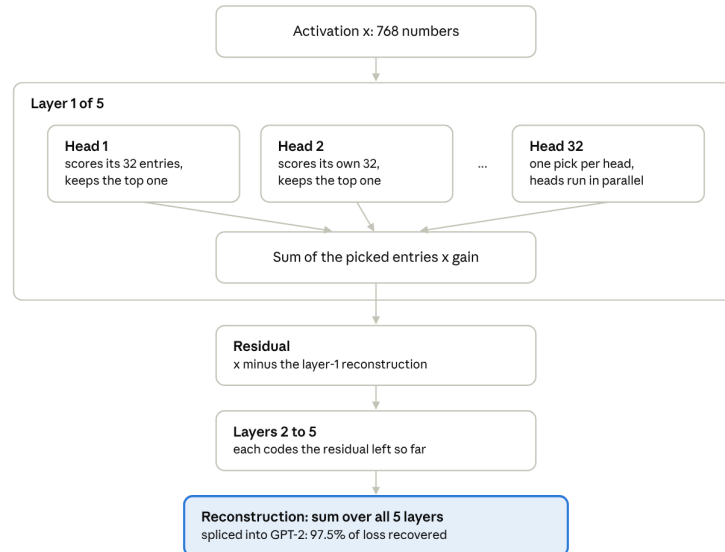
WHAT IT ASSUMES, AND WHY THAT IS ACCEPTABLE

1. **A representation is mostly a sum of a few categorical choices, plus a remainder that later rounds describe.** Much of language is categorical — which token, which syntactic role, which topic — and describing a remainder in successive rounds is how residual vector quantization works in compression [8]. On GPT-2, 800 bits per token rebuild the held-out state well enough that, put back into the model, it recovers 97.5% of the next-token loss that replacing the state with its average would cost.
2. **The classifiers act as roughly independent factors.** Independence is what lets a few small choices cover many combinations. On GPT-2 the information shared between two classifiers' choices is 0.27 bits in the first round and 0.02 bits by the third, against about 5 bits in each choice.
3. **The network tolerates running on the description.** If it did not, editing the description would mean nothing. Beyond the 97.5% recovery, swapping one classifier's choice changes GPT-2's next-token distribution by only 0.01–0.06 nats.
4. **The categories can be named.** Partly true. On GPT-2, a judge model shown a description of a first-round category picks out its most typical members from distractors 85–100% of the time — but only 55–69% for randomly chosen members, and at chance in the fourth and fifth rounds. On sentence embeddings, the strongest classifier sorts by topic and genre (crime news, legislation, encyclopedic description, and so on).
5. **The categories are a property of the data, not of the training run.** This has *not* held. Two runs from different random starts reconstruct equally well, but on both representations only the first round's classifiers recur; later rounds differ from run to run.

HOW IT WORKS, INTUITIVELY

Each layer picks one entry per head; later layers code what is left

5 layers x 32 heads x 32 entries = 800 bits per token, plus one gain per layer



Think of describing a token by answering a fixed list of multiple-choice questions, where the questions and the answers are learned. In the first round, 32 classifiers each pick their best category in parallel, and the picked vectors are summed and scaled to the size of what they explain. The next round sees only what the first got wrong, and so describes finer distinctions under the coarse ones; five rounds in all. Training uses hard choices in the forward pass and soft ones when computing gradients (a straight-through estimator). The design is a residual product quantizer [9, 8] — the compression literature’s closest relative — used not for compression but so that every choice can be read and swapped.

HOW IT COMPARES WITH EXISTING METHODS

Against supervised steering — the comparison that matters. Three tests so far, none of them yet the decisive one.

- *Swaps are specific (GPT-2).* Swapping a token’s category from j to j' moves the model’s output toward j' ’s typical output: among 31 alternatives, j' ranks 1.4–4 on average, and still 2–6 at four times the step, where a random step of the same size ranks 10–16. That is specificity against an internal control, not success at a named concept.
- *Language (sentence embeddings).* Forcing one of the method’s categories moved decoded text toward a target language 0.64 times as far as the supervised

difference-of-means vector did, and 4 times as far as a linear-probe direction. All methods, supervised included, damaged unrelated content about as much as a random step of the same size.

- *Reaching its own categories (sentence embeddings)*. For each category we fitted the two standard supervised vectors — difference of means and a linear probe — using the method’s own assignments as labels (a generous baseline: it already knows the answer), and gave every direction the same step. The soft version’s own direction reached its category on 75% of inputs, against 46–47% for the supervised vectors; the hard multi-round version’s reached only 37%, against 91–94%, so its categories are reachable but its own direction does not point at them.

The decisive test — steering toward externally defined concepts, with supervised vectors fitted to those concepts’ labels, SAE features and the method’s swaps scored by the same judge at matched side effects, as in AxBench — has not been run.

Against SAEs on steering. On sentence embeddings, at each model’s own step size, a feature of a standard SAE (32 active per input) takes effect on 46–50% of inputs and the multi-round method on 8–19%, both disturbing other features or classifiers about as much as a random step; a one-round version of the method takes effect on 41% while disturbing 1.4% of its other classifiers, against 18% for a random step. On GPT-2 only removal has been compared: setting one classifier to its average choice everywhere costs 0.07 nats, removing one SAE feature where active 0.055.

Against SAEs on reconstruction (a precondition, not the goal). Steering through a description only means something if the model runs on it.

GPT-2 small, after 8 blocks, held out		per-token code	unexplained variance	loss recovered
Ontologizer, 32 categories	800 bits		0.186	97.5%
Ontologizer, 64 categories	960 bits		0.164	97.9%
SAE, 32 active features	32 indices + 32 numbers		0.189	96.2%
SAE, 128 active features	128 indices + 128 numbers		0.116	98.6%

At nearly the same unexplained variance the method recovers more of the model’s loss than the matching SAE, so its errors fall where the model cares less. On sentence embeddings, 1900 bits of choices leave 15% of the variance unexplained, against 26% for an SAE at similar bits that also transmits 160 numbers.

WHAT WE CAN CLAIM

On both representations the method describes the network’s state with a small set of discrete choices about as efficiently as a sparse autoencoder of matched code size, or more so, and on GPT-2 the model runs on that description with little

loss. Swapping one choice moves the model’s output specifically toward the new category. Some categories can be named: on GPT-2 a first-round category’s most typical members, and — more promisingly — combinations of two first-round categories, which a judge tells apart from the rest of the category 77–96% of the time, against 48–50% for random splits of the same size; on sentence embeddings, a topic-and-genre classifier that recurs in every run. Whether the categories are in general easier to interpret than SAE features, and whether swapping them steers better than supervised vectors toward concepts a user cares about, is not established.

WHAT DOES NOT WORK YET

- **The classifiers past the first round are not reproducible.** Two runs agree on almost none of them, on both representations. A classifier that differs on every run cannot be named.
- **Categories are named from their extremes.** Descriptions fit a category’s most typical members, not its ordinary ones, and the deeper rounds cannot yet be named at all — though they are causal and steer specifically.
- **Steering through the input is not targeted.** On sentence embeddings, pushing an input toward one category changes other classifiers’ choices about as much as a random push of the same size. Swaps made on the description itself do not have this problem.

NEXT

The test the method needs is AxBench-style steering toward externally defined concepts: difference-of-means vectors from labeled pairs, SAE features and the method’s swaps, scored for concept success by a judge at matched side effects and fluency, plus detection against a linear probe. That benchmark targets Gemma 2, which would mean moving off GPT-2. In parallel: making the classifiers reproducible, or identifying the ones that are, and testing whether combinations of categories — rather than single ones — are the right unit for naming the deeper rounds.

REFERENCES

- [1] Alec Radford et al. *Language Models are Unsupervised Multitask Learners*. Tech. rep. OpenAI, 2019.
- [2] Paul-Ambroise Duquenne, Holger Schwenk, and Benoît Sagot. *SONAR: Sentence-Level Multimodal and Language-Agnostic Representations*. 2023. arXiv: 2308.11466 [cs.CL]. URL: <https://arxiv.org/abs/2308.11466>.

- [3] Alexander Matt Turner et al. *Steering Language Models With Activation Engineering*. 2024. arXiv: 2308.10248 [cs.CL]. URL: <https://arxiv.org/abs/2308.10248>.
- [4] Nina Rimskey et al. *Steering Llama 2 via Contrastive Activation Addition*. 2024. arXiv: 2312.06681 [cs.CL]. URL: <https://arxiv.org/abs/2312.06681>.
- [5] Trenton Bricken et al. “Towards Monosemanticity: Decomposing Language Models With Dictionary Learning”. In: *Transformer Circuits Thread* (2023). URL: <https://transformer-circuits.pub/2023/monosemantic-features/index.html>.
- [6] Leo Gao et al. *Scaling and evaluating sparse autoencoders*. 2024. arXiv: 2406.04093 [cs.LG]. URL: <https://arxiv.org/abs/2406.04093>.
- [7] Zhengxuan Wu et al. *AxBench: Steering LLMs? Even Simple Baselines Outperform Sparse Autoencoders*. 2025. arXiv: 2501.17148 [cs.CL]. URL: <https://arxiv.org/abs/2501.17148>.
- [8] Neil Zeghidour et al. *SoundStream: An End-to-End Neural Audio Codec*. 2021. arXiv: 2107.03312 [cs.SD]. URL: <https://arxiv.org/abs/2107.03312>.
- [9] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. “Product Quantization for Nearest Neighbor Search”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33.1 (2011), pp. 117–128.